

CH SHARIF AND SAEED HOSPITAL

Complete Backend Implementation Roadmap & Claude Prompts

Strictly mapped to Client Source of Truth (16 Client.pdf / 17 System Specifications)

Architecture: Node.js + Express.js + TypeScript + PostgreSQL (Prisma ORM)

Hidayat Baraye Istemal (How to use with Claude):

In prompts ko aik aik kar ke (Phase 1 se Phase 8) Claude ko dein. Har phase ke baad code verify karein, phir agla phase dein. Is se backend bina kisi hardcoded data ke clean aur scalable banega.

PHASE 1

Project Setup, Architecture & Database Schema (Prisma + PostgreSQL)

Objective: Create hms-backend directory, install packages, configure TS, and write complete Prisma schema strictly matching 16 Client.pdf.

TASK: Initialize Node.js + Express + TypeScript Backend for CH Sharif & Saeed Hospital HMS

Context:
We have an existing React 19 + TypeScript frontend inside 'ch-sharif-and-saeed-hospital---hms'.
Now build a modular, production-grade backend inside a new directory 'hms-backend'.
All business rules must strictly follow '16 Client.pdf' (Source of Truth v6.3).

Tech Stack:

- Runtime: Node.js (v20+)
- Framework: Express.js with TypeScript
- ORM: Prisma ORM with PostgreSQL
- Validation: Zod
- Auth: JWT (jsonwebtoken) + bcryptjs
- Utilities: cors, helmet, morgan, dotenv, express-rate-limit

Instructions:

1. Initialize 'hms-backend' with package.json, tsconfig.json, .env.example.
2. Structure folders:
src/config, src/controllers, src/middleware, src/routes, src/services, src/types, src/utils, app.ts, server.ts
3. In prisma/schema.prisma, define tables:
 - Staff (HR identity) vs PortalUser (Login credentials, 8 roles: SUPER_ADMIN, ADMIN, FRONT_DESK_BILLING, ADMISSION, INVENTORY_MANAGEMENT, PHARMACY_SUPER_ADMIN, PHARMACY_MANAGER, PHARMACY_SALES_DISPENSING).
 - Setup: HospitalProfile, Department, ServiceRate, Ward, Room, Bed, CorporatePanel.
 - Patient: PanelPatient (Permanent) vs SelfPayEncounter (Temporary case identity).
 - Billing: Appointment, HospitalInvoice, InvoiceLineItem, PaymentReceipt, UserCashBalance, AccountSettlement.
 - Admission: AdmissionRecord, BedTransferHistory, MedicationModeHistory (SELF vs HOSPITAL_MANAGED), AdmissionPaymentRequest, DualDischargeClearance.
 - Inventory: Supplier, StockItem, PurchaseOrder, StockLedger, SupplierLedger, DepartmentRequisition.
 - Pharmacy: MedicineMaster, MedicineBatch (FEFO), MedicineStockLedger, PharmacyDispense, PharmacyClearance.
 - Workforce: BiometricRawPunch, AttendanceRecord, AttendanceCorrectionLog, SalarySlip, DoctorCommissionRule, DoctorCommissionAccrual, CommissionPayout.
 - Audit trail on every transaction: created_by_user_id, collected_by_user_id, approved_by_user_id.
4. Create prisma/seed.ts with initial hospital profile, clinical/admin departments, wards/beds, and initial protected Super Admin & Admin accounts with hashed passwords.

PHASE 2**Authentication, Protected Account Governance & Cashier Account Settlement**

Objective: Implement JWT login, enforce Super Admin protection server-side, and implement exact-user cash custody settlement.

TASK: Implement Authentication, Protected Account Governance, and Universal User Cash Settlement API

Context:

Per '16 Client.pdf', two critical financial and security rules must be strictly enforced on backend:

1. Protected Governance: Admin cannot edit, demote, delete, or reset password of Super Admin.
2. Universal Cash Accountability: Admission portal never receives cash. Every cash user (Billing, funded Inventory, Pharmacy cashier) has their own UserCashBalance and submits their own AccountSettlement.

Instructions:

1. Implement JWT Auth: POST /api/v1/auth/login, GET /api/v1/auth/me, POST /api/v1/auth/refresh.
2. Create authenticate and authorizeRoles(...roles) middlewares.
3. In User Management controller, enforce protected account rules:
 - If user role is ADMIN and target user is SUPER_ADMIN -> return 403 Forbidden.
 - Prevent deleting currently active Super Admin or last remaining Super Admin.
4. Universal Cash Accountability Endpoints:
 - GET /api/v1/finance/my-balance-sheet: Returns logged-in cashier's opening float, collections, advances, refunds, petty expenses, and expected physical cash.
 - POST /api/v1/finance/settlements/submit: Cashier submits counted physical cash, calculates variance against unsettled transactions, and submits for custody transfer.
 - GET /api/v1/finance/settlements/pending: For Super Admin/Admin to review.
 - POST /api/v1/finance/settlements/:id/action: Reviewer accepts, partially accepts, or returns settlement.

PHASE 3**Hospital Masters, Staff 360° Profiles & Patient Registry Endpoints**

Objective: Build real CRUD APIs for Hospital Setup, Staff Master 360° view, and Pakistani CNIC/Phone patient registry.

TASK: Build REST APIs for Hospital Setup, Staff 360° Profiles, and Patient Registry

Context:

Refer to '16 Client.pdf' Sections 4, 5, 6, 31, 32. Eliminate all client-side mock arrays.

Instructions:

1. Hospital Setup APIs:
 - GET/PUT /api/v1/setup/hospital-profile (Hospital details, logo, NTN, print headers/footers).
 - /api/v1/setup/departments (Clinical & administrative).
 - /api/v1/setup/services-rates (Standard rates, discount allowances, panel applicability).
 - /api/v1/setup/wards-rooms-beds (Department -> Ward -> Room -> Bed hierarchy; status: Available, Occupied, Maintenance).
 - /api/v1/setup/corporate-panels (Corporate panel organizations, discount rules, credit limits).
2. Staff 360° Profile API:
 - /api/v1/staff (CRUD).
 - GET /api/v1/staff/:id/360: Returns HR profile, linked portal user account, shift history, attendance summary, staff-specific salary profile, and doctor commission rules.
3. Patient Registry API:
 - Validate Pakistani CNIC (XXXXX-XXXXXXX-X) and phone (03XX-XXXXXXX).
 - GET /api/v1/patients/check-duplicate?cnic=...&phone=...
 - Permanent Panel Patients (/api/v1/patients/panel).
 - Temporary Self-Pay Encounters (/api/v1/patients/encounters).

PHASE 4**Front Desk Billing, Appointments & Doctor Share Calculation Engine**

Objective: Build appointment flow, advance collection, OPD/Emergency invoicing, and Doctor Commission calculation.

TASK: Build Front Desk Billing, Appointments, and Invoicing Calculation Engine

Context:

Refer to '16 Client.pdf' Section 7, 8, 22, 35. Front Desk is the ONLY portal collecting hospital patient cash.

Instructions:

1. Appointments Flow:
 - POST /api/v1/appointments: Book appointment for panel or self-pay.
 - POST /api/v1/appointments/:id/advance: Collect advance payment; generates receipt linked to cashier.
 - POST /api/v1/appointments/:id/check-in: Converts appointment to active OPD/Observation/Emergency encounter.
2. Invoicing & Doctor Commission Calculation Engine:
 - Formula:
 - Net Service Amount = Gross Service Amount - Approved Discount
 - Doctor Commission = Net Eligible Service Amount * Doctor Commission % (or fixed rate)
 - Hospital Remaining Share = Net Service Amount - Doctor Commission
 - Automatically apply Panel discount rules if panel patient.
 - Require Admin approval if discount exceeds threshold.
3. Receipts & Cashier Updates:
 - Adjust prior advance payment against final hospital invoice.
 - Cash payments automatically update current cashier's UserCashBalance.
 - Digital/POS payments tracked separately without inflating physical cash.

PHASE 5

Inpatient Admission, Bed Lifecycle & Dual Clearance Discharge Workflow

Objective: Manage planned admission, bed transfers, medication modes, and the 3-key gate for final discharge.

TASK: Implement Inpatient Admission (IPD), Bed Allocation, and Dual Clearance Discharge

Context:

Refer to '16 Client.pdf' Sections 9, 10, 11, 12, 13, 23.

Crucial: Admission NEVER collects cash. Final discharge requires Dual Clearance.

Instructions:

1. Planned Admission & Billing Hand-off:
 - POST /api/v1/admissions/planned: Create planned admission.
 - POST /api/v1/admissions/:id/request-advance: Sends payment request to Billing queue.
2. Bed Lifecycle:
 - POST /api/v1/admissions/:id/check-in: Assign bed -> Status becomes Occupied.
 - POST /api/v1/admissions/:id/bed-transfer: Transfer patient bed; log from_bed, to_bed, timestamp, actor.
 - POST /api/v1/admissions/:id/add-service: Append running hospital services and bed day charges.
3. Medication Mode:
 - POST /api/v1/admissions/:id/medication-mode: Toggle SELF vs HOSPITAL_MANAGED with reason and actor.
 - In HOSPITAL_MANAGED mode, allow sending medicine requests to Standalone Pharmacy queue.
4. Dual Clearance Discharge Gate:
 - Clinical Ready = Doctor sign-off.
 - Hospital Clearance = Billing confirms all hospital bills paid.
 - Pharmacy Clearance = Pharmacy confirms all medicine dues cleared.
 - POST /api/v1/admissions/:id/discharge: Allowed ONLY when all 3 clearances are approved. Automatically frees bed to Available.

PHASE 6

General Inventory & Standalone Pharmacy Integration Engine

Objective: Implement non-medicine inventory ledgers, and batch/FEFO dispensing with Pharmacy clearance.

TASK: Implement General Inventory Ledgers and Standalone Pharmacy Integration

Context:

Refer to '16 Client.pdf' Sections 13, 14, 16, 17, 33, 34.

HMS Inventory is non-medicine; Standalone Pharmacy owns medicine stock and FEFO dispensing.

Instructions:

1. General Inventory:
 - Supplier Master & Supplier Ledger (track purchases, returns, payments, credits, outstanding).
 - Fund Request: Inventory staff requests petty cash advance -> Admin approves -> credited to user cash balance.
 - Purchase/Receipt: Posts stock IN into StockLedger; cash payment reduces user cash balance.
 - Department Issue/Return: Requisition -> Issue stock -> Deduct StockLedger with department tag.
2. Standalone Pharmacy Integration:
 - Medicine Master, Batches, Expiry, Cost, Sale Price.
 - FEFO Engine: Auto-allocate nearest non-expired batch upon dispensing.
 - Dispensing: Retail walk-in sale + HMS Inpatient Request queue.
 - Dispensing creates Pharmacy Invoice and updates Pharmacy Clearance callback to HMS Admission.

PHASE 7

Biometric Attendance, Salary Payroll & Doctor Commission Engines

Objective: Process punch logs, calculate salary payroll, and independently accrue/pay doctor commissions.

TASK: Implement Attendance Normalization, Staff Salary Payroll, and Doctor Commission Streams

Context:

Refer to '16 Client.pdf' Sections 18, 19, 20, 21, 25, 26, 35.

Crucial: Salary Payroll (attendance-driven) and Doctor Commission (service-driven) are independent liabilities!

Instructions:

1. Biometric Attendance Engine:
 - POST /api/v1/attendance/punch-ingest: Receive raw device punch logs.
 - Map punches against assigned shift (grace period, late-in, early-exit, absent).
 - Admin Attendance Correction: PUT /api/v1/attendance/:id/correct (log original, corrected, reason, user).
2. Salary Payroll Engine:
 - Formula:

$$\text{Generated Salary} = \text{Base Salary} + \text{Approved Allowances} - \text{Attendance Deductions} - \text{Other Deductions} +/- \text{Adjustments}$$
 - Lifecycle: Generated -> Approved -> Paid (Full, Partial, or Later).
3. Doctor Commission Engine:
 - Accrues from completed eligible service lines.
 - Single-inclusion rule: A service line can only be included in ONE generated commission statement.
 - Generation -> Approval -> Independent payout and outstanding tracking.

PHASE 8

Frontend Refactoring (Removing LocalStorage & Connecting Real Express APIs)

Objective: Replace localStorage in the React frontend with Axios client and connect live endpoints.

TASK: Refactor React Frontend to Remove LocalStorage Mock Data and Connect to Live Express Backend

Context:

Frontend in 'ch-sharif-and-saeed-hospital---hms' uses localStorage (hms_patient_registry_v1, etc.) and ModulePlaceholderView.

Connect it to the backend running at http://localhost:5000/api/v1.

Instructions:

1. Create Axios client in src/services/api/apiClient.ts:
 - Base URL from VITE_API_URL or http://localhost:5000/api/v1.
 - Request interceptor: Attach JWT Bearer token from AuthContext.
 - Response interceptor: Handle 401 (redirect to /login), 403, and toast notifications.
2. Refactor services in src/services/:
 - Replace adminUserService.ts, patientRegistryService.ts, departmentService.ts, etc., with real async HTTP calls.
 - Remove hardcoded mock arrays and localStorage keys.
3. Replace ModulePlaceholderView.tsx with real working components:
 - Front Desk Invoicing & Payment Collection screen.
 - Cashier Balance Sheet & Account Settlement modal.
 - Admission Bed Allocation & Dual Clearance screen.
 - Inventory Requisition & Stock Ledger screen.
 - Standalone Pharmacy Queue & Dispensing screen.